RPOO – Corrigés expliqués

TP1 à TP6 · Master 1 Informatique · cours d'Alain Plantec

Ce document reprend tes corrigés Java, méthode par méthode : **principe**, **complexité**, **cas limites**, **pièges**. Un chapitre à part explique comment fonctionnent les tests **JUnit** (utilisés dans tous les TP). Les comportements signalés « Bug vérifié » ont été exécutés pour de vrai (JDK 21), ce ne sont pas des suppositions.

> **Non fourni.** Dans les fichiers reçus, il n'y a pas de `ClosedHashDict.java` (table de hash fermée, sujet TP3), pas de `ArrayQueue.java` (TP6), pas de `ArrayTest` ni de `copyWithout` (TP1). Pour ces points j'explique le sujet et ce que les tests imposent, sans inventer ton code. Envoie-les et je complète.

Sommaire

0. Les tests JUnit : comment ça marche **4**

0.1 L'idée . . . 4
0.2 Anatomie d'un test (modèle Arrange – Act – Assert) . . . 4
0.3 Les assertions à connaître . . . 4
0.4 JUnit 4 ou JUnit 5 ? Tes TP utilisent les deux . . . 4
0.5 Chaque test a son propre objet . . . 5
0.6 Tester les exceptions . . . 5
0.7 Tester ce qui est protected ou invisible . . . 5
0.8 Lancer les tests dans Eclipse . . . 5
0.9 Écrire un bon test : checklist . . . 5
0.10 Quatre tests de tes TP qui ne testent pas ce qu'ils annoncent . . . 6

1. TP1 – Tableau de caractères : CharArray **7**

1.1 Constructeurs, size, get, set . . . 7
1.2 toString et indexOf . . . 7
1.3 concat . . . 7
1.4 containsSubStartingAt et containsSub . . . 8
1.5 copyFromTo et trim . . . 8
1.6 Les tests CharArrayTest (JUnit 4) . . . 9

2. TP2 – Tableau dynamique : DynTab **10**

2.1 Représentation et invariants . . . 10
2.2 Gestion de la capacité . . . 10
2.3 add(index, element) . . . 10
2.4 Lecture : get, indexOf, lastIndexOf, contains . . . 11
2.5 Suppression . . . 11
2.6 clear, toArray, equals . . . 12
2.7 Variante : DynTab2 (sentinelle) . . . 12
2.8 Les tests DynTabTest (JUnit 4) . . . 13

3. TP3 – Tables associatives : AssocTable et table de hash fermée **14**

3.1 Le contrat Map en deux lignes . . . 14
3.2 AssocTable : une ArrayList de couples . . . 14
3.3 La table de hash fermée (principe du sujet) . . . 15
3.4 Les tests AssocTableTest (JUnit 5) . . . 16

4. TP4 – Table de hash ouverte : OpenedHashDict **17**

4.1 Structure et position . . . 17
4.2 put, get, remove . . . 17

4.3 Taille, parcours . . . 18
4.4 Les vues keySet, values, entrySet et leurs caches . . . 18
4.5 Autres remarques . . . 19
4.6 Les tests OpenedHashDictTest (JUnit 5) . . . 19

5. TP5 – Ensemble : MonEnsemble et MonIterateur 20

5.1 Représentation . . . 20
5.2 contains, add, grow . . . 20
5.3 remove, toArray, compact, toString . . . 20
5.4 Trois bugs vérifiés dans MonEnsemble . . . 22
5.5 MonIterateur et l'exercice 2 . . . 22
5.6 Les tests (JUnit 4) . . . 23

6. TP6 – File circulaire : ArrayQueue 24

6.1 Les deux interfaces . . . 24
6.2 Le tableau circulaire (table 1 du sujet) . . . 24
6.3 Ce que les tests imposent . . . 24
6.4 Squelette possible (mon illustration, pas ton code) . . . 24
6.5 ArrayQueueTest (JUnit 5) : lecture rapide . . . 25

7. Récapitulatif des points à corriger 26

0. Les tests JUnit : comment ça marche

0.1 L'idée

Un test unitaire est une petite méthode qui **appelle ton code, puis vérifie le résultat**. JUnit trouve toutes les méthodes marquées `@Test`, les exécute une par une et affiche **vert** (toutes les vérifications passent) ou **rouge** (au moins une échoue). Pas de `main` : c'est JUnit qui lance les tests, comme le demande le sujet du TP2.

0.2 Anatomie d'un test (modèle Arrange – Act – Assert)

```
@Test
public void testRemoveInt2() {
    DynTab<String> dl = new DynTab<String>();   // 1. Arrange : préparer l'objet
    dl.add("A");
    dl.add("B");
    dl.remove(0);                               // 2. Act : exécuter l'action testée
    assertTrue(dl.size() == 1);                 // 3. Assert : vérifier le résultat
    assertTrue(dl.get(0).equals("B"));
}
```

Si une assertion est fausse, elle lève une `AssertionError` : le test s'arrête là et JUnit le marque en échec. Si le code lève une exception **inattendue**, le test est en « erreur » (autre couleur dans Eclipse, mais même résultat : rouge).

0.3 Les assertions à connaître

Assertion	Vérifie que…	Remarque
`assertTrue(c)` / `assertFalse(c)`	la condition est vraie / fausse	Message d'échec pauvre : « expected true »
`assertEquals(attendu, obtenu)`	`attendu.equals(obtenu)`	Ordre **attendu d'abord** ; message « expected:<3> but was:<2> »
`assertSame(a, b)`	`a == b` (même objet)	À utiliser à la place de `assertTrue(a == b)`
`assertNull(x)` / `assertNotNull(x)`	x vaut / ne vaut pas `null`	
`assertThrows(Classe.class, () -> {...})`	le code lève cette exception	JUnit 5 (et JUnit 4.13+)
`fail("msg")`	échoue toujours	Pour marquer un chemin « ne doit pas arriver »

Conseil pratique : préfère `assertEquals(2, dl.size())` à `assertTrue(dl.size() == 2)`. Le test vérifie la même chose, mais en cas d'échec JUnit t'affiche la valeur obtenue au lieu de seulement « false ».

0.4 JUnit 4 ou JUnit 5 ? Tes TP utilisent les deux

	JUnit 4 (TP1, TP2, TP5)	JUnit 5 / Jupiter (TP3, TP4, TP6)
Import du @Test	`org.junit.Test`	`org.junit.jupiter.api.Test`
Import des assertions	`org.junit.Assert.*`	`org.junit.jupiter.api.Assertions.*`
Avant / après chaque test	`@Before` / `@After`	`@BeforeEach` / `@AfterEach`
Une fois pour toute la classe	`@BeforeClass` / `@AfterClass`	`@BeforeAll` / `@AfterAll`
Exception attendue	`@Test(expected = X.class)`	`assertThrows(X.class, () -> ...)`
Visibilité des méthodes	`public` obligatoire	package-private suffit (`void size()`)
Message optionnel	en **premier** argument	en **dernier** argument

> **Piège.** Ne mélange jamais les imports des deux versions dans le même fichier : un `@Test` de JUnit 4 est ignoré par le moteur JUnit 5, et inversement. Reconnais la version à l'import (`org.junit.` ou `org.junit.jupiter.`).

0.5 Chaque test a son propre objet

JUnit crée une **nouvelle instance de la classe de test pour chaque méthode** `@Test`. Dans `ArrayQueueTest`, le champ `private ArrayQueue<Integer> myq = new ArrayQueue<Integer>(deuxmille);` donne donc une file neuve et vide à chaque test. Conséquences : les tests ne doivent **pas dépendre les uns des autres**, et l'**ordre d'exécution n'est pas garanti**. Pour préparer un objet commun, on peut aussi utiliser `@Before` / `@BeforeEach`.

0.6 Tester les exceptions

Il y a trois façons, de la moins bonne à la meilleure :

```
// (a) try/catch à la main : ce que font DynTabTest et CharArrayTest
try {
    t.set(2, 'c');
    assertFalse(true);                          // atteint seulement si AUCUNE exception
} catch (IndexOutOfBoundsException e) {
    assertTrue(true);                           // ne vérifie rien
}

// (b) JUnit 4 : l'annotation
@Test(expected = IndexOutOfBoundsException.class)
public void testSet2() { new CharArray("AB").set(2, 'c'); }

// (c) JUnit 5 : assertThrows (le plus lisible, et on peut garder l'exception)
assertThrows(NoSuchElementException.class, () -> { myq.remove(); });
```

> **À retenir.** Pourquoi `assertFalse(true)` dans le `try` fonctionne : il lève une `AssertionError`, qui est une `Error` et non une `Exception`. Le `catch (Exception e)` ne l'attrape donc pas et le test échoue bien. Mais `assertTrue(true)` dans un `catch` est inutile : il ne peut jamais échouer.

0.7 Tester ce qui est `protected` ou invisible

- **Même package** (TP1, TP2, TP5, tous dans le package par défaut) : `protected` donne aussi l'accès au package, donc le test appelle directement `capacity()` ou lit `nbPlacesLibres`.
- **Autre package** (TP6 : les tests sont dans `tests`, la classe dans `arrayqueue`) : on écrit une sous-classe de test qui expose les champs, c'est le début de `ArrayQueueTest` :

```
class ArrayQueue<E> extends arrayqueue.ArrayQueue<E> {     // sous-classe, dans le package "tests"
    public ArrayQueue(int capacity) { super(capacity); }
    int tete()         { return this.tete; }                // une sous-classe a le droit de lire
    int queue()        { return this.queue; }               // les champs protected de son parent
    Object[] tableau() { return this.tableau; }
}
```

0.8 Lancer les tests dans Eclipse

- Créer : clic droit sur la classe → **New → JUnit Test Case**, choisir JUnit 4 ou « New JUnit Jupiter test », indiquer la classe à tester. Si Eclipse propose « Add JUnit … library to the build path » : accepter.
- Lancer : clic droit sur la classe de test → **Run As → JUnit Test** (ou sur une seule méthode pour n'exécuter que celle-là).
- Lire : la vue **JUnit** montre une barre verte ou rouge. Clique sur un test rouge : le panneau « Failure Trace » donne le message et la **ligne exacte** de l'assertion qui a échoué.
- Démarche TDD simple : écrire le test d'abord (il est rouge), coder le minimum pour qu'il passe (vert), puis améliorer.

0.9 Écrire un bon test : checklist

- Tester **le cas normal** puis **les bords** : structure vide, un seul élément, pleine, premier / dernier indice, indice `-1` et `size()`, valeur `null`, doublons.
- Un test = une idée, avec un nom qui dit laquelle (`addRemoveIsFull`, plutôt que `test3`).
- Un test doit pouvoir **échouer** : méfie-toi des assertions toujours vraies (`assertTrue(true)`).
- Comparer des objets avec `assertEquals`, pas ==. Sur des `Integer`, == ne marche que pour -128..127 (cache) ou si c'est littéralement le même objet.

0.10 Quatre tests de tes TP qui ne testent pas ce qu'ils annoncent

Test	Problème	Correction
`DynTabTest.testAdd3` et `testAdd7`	Si `add(-10, …)` ne lève rien, le `try` passe quand même (`assertTrue(true)`)	`@Test(expected = IndexOutOfBoundsException.class)` ou `assertThrows`
`ArrayQueueTest.size`	La boucle `for (int x = currentSize; x <= mille; x--)` ne s'exécute jamais : `currentSize` vaut 1010, et `1010 <= 1000` est faux	Boucler `while (currentSize > 0)` en vérifiant `size()` après chaque `poll()`
`MonEnsembleTest.testCompact`	N'appelle jamais `compact()` : il teste seulement `toArray()`	Voir le test proposé au chapitre 5
`MonIterateurTest.concurrentModificationExceptionTest`	Le `catch` est **dans** la boucle : après l'exception, `hasNext()` reste vrai et la boucle ne finit jamais	`assertThrows` autour de la boucle (chapitre 5)

1. TP1 – Tableau de caractères : `CharArray`

Demandé : une classe qui met en œuvre l'interface `Tableau` et ajoute `concat`, `containsSub`, `copyFromTo`, `copyWithout` et `trim`. Le corrigé reçu est `CharArray`, un tableau de `Character` (le sujet parle d'`Array` et de `null` ; ici, `trim` enlève les **espaces**).

```
public interface Tableau<E> {
    int size();
    E get(int index);
    void set (int index, E c);
    int indexOf(E c);
}
```

1.1 Constructeurs, `size`, `get`, `set`

```
public CharArray(String s) {
    t = new Character[s.length()];
    for (int i = 0; i < s.length(); i++)
        t[i] = s.charAt(i);
}

private CharArray(int s) {
    t = new Character[s];
}
```

Le premier constructeur copie la chaîne caractère par caractère dans un `Character[]`. Le second est **privé** : il sert uniquement aux méthodes internes (`copyFromTo`) à fabriquer un tableau de la bonne taille rempli de `null`. `size()` renvoie `t.length` : la taille est fixe, un `CharArray` n'est pas redimensionnable.

`get` et `set` ne vérifient pas l'indice : la JVM lève elle-même `ArrayIndexOutOfBoundsException`, sous-classe de `IndexOutOfBoundsException`. C'est pourquoi les tests attendent `IndexOutOfBoundsException`.

1.2 `toString` et `indexOf`

```
public String toString() {
    String s = "";
    for (int i = 0; i < this.size(); i++) {
        s += get(i);
    }
    return s;
}

public int indexOf(Character c) {
    for (int i = 0; i < this.size(); i++) {
        if (c.equals(t[i]))
            return i;
    }
    return -1;
}
```

> **Amélioration.** `s += get(i)` recrée une chaîne à chaque tour : coût en O(n²). Avec `StringBuilder sb = new StringBuilder(); … sb.append(get(i)); … return sb.toString();` on passe en O(n).

`indexOf` compare avec `equals` (et non `==`, qui comparerait des références de `Character`). Écrire `c.equals(t[i])` lève une `NullPointerException` si `c` est `null` ; `Objects.equals(c, t[i])` l'évite. Complexité O(n), renvoie `-1` si absent.

1.3 `concat`

```
public CharArray concat(CharArray ch2) {
    return new CharArray(this.toString()+ch2.toString());
}
```

Astucieux et court : on passe par les chaînes. Dans les tests, `"ABC".concat("")` doit contenir A, B, C. Alternative sans conversion en `String` :

```
public CharArray concat(CharArray o) {
    CharArray r = new CharArray(this.size() + o.size());
    for (int i = 0; i < size(); i++)    r.set(i, get(i));
    for (int i = 0; i < o.size(); i++)  r.set(size() + i, o.get(i));
    return r;
}
```

1.4 `containsSubStartingAt` et `containsSub`

```
public Boolean containsSubStartingAt(CharArray sub, int start) {
    if (start < 0 || start >= this.size()) {
        throw new ArrayIndexOutOfBoundsException(start);
    }
    int myidx = start;
    int subidx = 0;
    while (myidx < this.size() && subidx < sub.size()) {
        if (! (get(myidx).equals(sub.get(subidx))) ) {
            return false;
        }
        myidx++;
        subidx++;
    }
    return subidx == sub.size();
}
```

Principe : deux curseurs avancent ensemble, `myidx` dans le receveur et `subidx` dans le motif. Au premier caractère différent → `false`. Si la boucle s'arrête parce que le receveur est fini avant le motif, `subidx < sub.size()` → `false`. Le test final `subidx == sub.size()` signifie « tout le motif a été retrouvé ».

Cas limites imposés par les tests : un motif vide est contenu partout (`true` pour start = 0, 1, 2 sur « ABC »), mais `start = 3` (hors du tableau) lève une exception : le contrôle de `start` se fait **avant** tout.

```
public Boolean containsSub(CharArray sub) {
    if (sub.size() == 0) return true;
    int start = indexOf(sub.get(0));
    if (start == -1) return false;
    return containsSubStartingAt (sub, start);
}
```

> **Bug vérifié.** `containsSub` ne teste que **la première occurrence** du premier caractère du motif. Résultat de l'exécution : `new CharArray("ABAC").containsSub(new CharArray("AC"))` renvoie **false**, alors que « AC » est bien aux indices 2-3. Les tests passent parce que « ABCABCAB » ne révèle pas le problème.

```
public Boolean containsSub(CharArray sub) {
    if (sub.size() == 0) return true;
    for (int start = 0; start + sub.size() <= this.size(); start++)
        if (containsSubStartingAt(sub, start)) return true;   // on essaie TOUTES les positions
    return false;
}
```

Test qui aurait détecté le bug : `assertTrue(new CharArray("ABAC").containsSub(new CharArray("AC")));`

1.5 `copyFromTo` et `trim`

```
public CharArray copyFromTo(int start, int stop) {
    if (start < 0 || start >= this.size()) {
        throw new ArrayIndexOutOfBoundsException(start);
    }
    if (stop < 0 || stop >= this.size()) {
        throw new ArrayIndexOutOfBoundsException(stop);
    }

    CharArray result = new CharArray(stop - start + 1);
    for (int i = 0; i < result.size(); i++) {
        result.set(i, this.get(i+start));
    }
    return result;
}
```

```
public CharArray trim() {
    int start = 0;
    while (start < this.size() && (this.get(start)).equals(' ')) {
        start ++;
    }
    if (start == this.size()) {
        return new CharArray(0);
    }
    int stop = this.size() - 1;
    while ((this.get(stop)).equals(' ')) {
        stop--;
    }
    return copyFromTo(start, stop);
}
```

`copyFromTo(start, stop)` copie les indices **start à stop inclus** (taille `stop - start + 1`). Les deux bornes sont contrôlées. Cas particulier testé : `copyFromTo(5, 4)` donne une taille 0, donc la chaîne vide (copie « vide » volontairement permise).

`trim` avance `start` tant que le caractère est un espace. Si on atteint la fin, tout était des espaces → tableau vide. Sinon `stop` recule depuis la fin ; cette seconde boucle est **sûre sans test de borne** car on sait qu'il existe au moins un caractère non-espace (celui trouvé par `start`). Puis `copyFromTo(start, stop)` fait le travail.

1.6 Les tests `CharArrayTest` (JUnit 4)

- `testCharArray1/2` : taille et capacité d'un tableau vide et d'un tableau à un élément.
- `testToString`, `testSet`, `testIndexOf1..3` : cas vide, premier et dernier élément.
- `testSet2`, `testContainsSubStartingAt0`, `testCopyFromTo` : exceptions attendues (try/catch, voir 0.6).
- `testTrim0..4` : vide, sans espace, espace devant, derrière, des deux côtés.

Manque : `copyWithout` (cité dans le sujet, absent du fichier reçu) et le cas `containsSub` ci-dessus.

2. TP2 – Tableau dynamique : `DynTab<E>`

Demandé : implémenter `DynList<E>` (sous-ensemble de `java.util.List`) avec un tableau interne **qui double de taille** quand il est plein, sans jamais réduire à la suppression.

2.1 Représentation et invariants

```
public DynTab () {
    tab = (E[]) new Object[1];
    s = 0;
}
```

- `tab` : le stockage (capacité = `tab.length`, démarre à **1**). `s` : le nombre d'éléments réellement présents.
- Invariants : `0 ≤ s ≤ tab.length`, et seules les cases `0 … s-1` sont significatives.
- `(E[]) new Object[1]` : Java interdit `new E[1]` (effacement de type). On crée un `Object[]` et on le convertit ; l'avertissement « unchecked » est sans danger tant que `tab` n'est jamais exposé hors de la classe.

2.2 Gestion de la capacité

```
protected void checkCapacity(int incr) {
    if (this.capacity() >= incr + this.size()) return;
    tab = (E[]) new Object[this.capacity() * 2];
    this.checkCapacity(incr);
}
```

« Il me faut de la place pour `incr` éléments de plus ? Si oui, rien à faire. Sinon j'alloue un tableau deux fois plus grand et je recommence » (récursion jusqu'à ce que ça suffise). **Attention : cette méthode n'y recopie rien.** La recopie est faite par `add`, qui garde l'ancien tableau dans `previous`.

Code exécuté	Taille	Capacité
`new DynTab()`	0	1
`add("A")`	1	1
`add("B")`	2	2 (1 plein → ×2)
`add("C")`	3	4
`add("D")`	4	4
`add("E")`	5	8
`remove("B")`, `remove("D")`, `remove("E")`	2	8 (inchangée)

C'est exactement ce que vérifie `testAdd1`.

2.3 `add(index, element)`

```
@Override
public void add(int index, E element) {
    E previous [] = tab;
    // IndexOutOfBoundsException - if the index is out of range (index < 0 || index > size())
    if (index < 0 || index > this.size()) throw new IndexOutOfBoundsException();
    this.checkCapacity(1);
    // ** ATTENTION ** On copie par la fin
    for (int i = this.size()-1; i >= index; i--) {
        tab[i+1] = previous[i];
    }
    tab[index] = element;
    for (int i = index - 1; i >= 0; i--)
        tab[i] = previous[i];
    s++;
}
```

Étapes : (1) contrôle `0 ≤ index ≤ size()` (insérer **à** `size()` = ajouter à la fin est permis, pas au-delà) ; (2) `checkCapacity(1)` peut remplacer `tab` ; (3) **décalage vers la droite en partant de la fin** ; (4)

pose de l'élément ; (5) recopie des éléments situés avant `index`.

Exemple, `tab = [A, B, C, _]`, `s = 3`, `add(1, "X")` (pas de réallocation) :

```
départ         [A][B][C][_]
i=2 : tab[3]=C [A][B][C][C]
i=1 : tab[2]=B [A][B][B][C]
tab[1] = X     [A][X][B][C]      → s = 4
```

> **Piège.** « On copie par la fin » : en décalant du début vers la fin, `tab[1]` écraserait `tab[2]` avant qu'on l'ait copié, et on recopierait la même valeur partout.

> **Amélioration.** La dernière boucle (`tab[i] = previous[i]`) tourne **à chaque appel**, même sans réallocation (alors `previous == tab` : elle copie une case sur elle-même). Un `add` en fin de tableau coûte donc O(n) au lieu de O(1) amorti. Version corrigée avec `System.arraycopy` (qui gère correctement le chevauchement) :

```
public void add(int index, E element) {
    if (index < 0 || index > size()) throw new IndexOutOfBoundsException();
    E[] previous = tab;
    checkCapacity(1);                                    // tab peut changer
    if (previous != tab)                                 // nouveau tableau : recopier le début
        System.arraycopy(previous, 0, tab, 0, index);
    System.arraycopy(previous, index, tab, index + 1, s - index);   // décaler la fin
    tab[index] = element;
    s++;
}
```

Pourquoi doubler et pas ajouter 1 ? Avec un doublement, n ajouts coûtent au total moins de 2n copies : le coût **amorti** d'un ajout en fin est O(1).

2.4 Lecture : `get`, `indexOf`, `lastIndexOf`, `contains`

```
@Override
public E get(int index) {
    // ** Attention ** il faut tester la borne sup
    // IndexOutOfBoundsException - if the index is out of range (index < 0 || index >= size())
    if (index >= this.size())
        throw new IndexOutOfBoundsException();
    return tab[index];
}
```

Le contrôle de la **borne supérieure** est indispensable : sans lui, `get(size())` lirait une case vide du tableau (capacité > taille) au lieu de lever une exception. L'indice négatif, lui, est attrapé par la JVM.

```
@Override
public int indexOf(Object o) {
    for (int pos = 0; pos < size(); pos++) {
        E item = tab[pos];
        if (item.equals(o))
            return pos;
    }
    return -1;
}
```

`indexOf` parcourt de 0 à `size()-1` avec `equals` ; `lastIndexOf` parcourt à l'envers ; `contains` = `indexOf > -1`. `item.equals(o)` plante si un élément stocké vaut `null` : utiliser `Objects.equals(item, o)`.

2.5 Suppression

```
@Override
public E remove(int index) {
    if (index < 0 || index >= this.size()) throw new IndexOutOfBoundsException();
    E element = tab[index];
    for (int i = index; i < this.size() -1; i++)
        tab[i] = tab[i+1];
    s--;
    return element;
}
```

```
@Override
public boolean remove(Object o) {
    int pos = indexOf(o);
    if (pos > -1) {
        this.remove(pos);
        return true;
    }
    return false;
}
```

- `remove(int)` décale tout ce qui est à droite d'un cran vers la gauche (O(n)), décrémente `s`, renvoie l'élément retiré. **Aucune réallocation**, comme demandé.
- `remove(Object)` cherche la position avec `indexOf`, puis délègue à `remove(int)` ; renvoie `false` si absent.
- Surcharge : `dl.remove("B")` appelle `remove(Object)`, `dl.remove(0)` appelle `remove(int)`. Sur un `DynTab<Integer>`, `dl.remove(5)` retire l'**indice** 5 ; pour retirer la valeur 5 : `dl.remove(Integer.valueOf(5))`.

> **Amélioration.** Après le décalage, `tab[s]` (l'ancienne dernière case) garde une référence vers l'élément : il ne sera pas ramassé par le GC. Ajouter `tab[s] = null;` après `s--`.

2.6 `clear`, `toArray`, `equals`

```
@Override
public void clear() {
    tab = (E[]) new Object[1];
    s = 0;
}
```

`clear` réalloue un tableau de capacité 1 : c'est ce que vérifie `testClear` (état identique à un `DynTab` neuf). `toArray` copie les `s` premiers éléments dans un `Object[]` de la bonne taille.

> **Bug vérifié.** `DynList` déclare `equals(Object)` mais `DynTab` ne le redéfinit pas : c'est donc `Object.equals` (identité) qui est utilisé. Deux `DynTab` contenant « A, B » ne sont pas « égaux ». Version conforme à `List` :

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof DynTab)) return false;
    DynTab<?> other = (DynTab<?>) o;
    if (size() != other.size()) return false;
    for (int i = 0; i < size(); i++)
        if (!java.util.Objects.equals(tab[i], other.tab[i])) return false;
    return true;
}
@Override
public int hashCode() {                      // toujours redéfinir hashCode avec equals
    int h = 1;
    for (int i = 0; i < s; i++) h = 31 * h + (tab[i] == null ? 0 : tab[i].hashCode());
    return h;
}
```

2.7 Variante : `DynTab2` (sentinelle)

```
public class DynTab2 {
    static class Sentinelle {}
    static Sentinelle FIN = new Sentinelle();
    Object [] l;

    public DynTab2 () {
        l = new Object [1];
        l[0] = FIN;
    }

    int size () {
        int s = 0;
        while (l[s] != FIN) s++;
        return s;
    }


}
```

Idée : au lieu d'un compteur `s`, on place un objet **sentinelle** `FIN` juste après le dernier élément. `size()` parcourt jusqu'à la sentinelle : O(n) au lieu de O(1). Avantage : une seule donnée à tenir à jour. Inconvénient : il faut toujours garder une case libre pour `FIN`, et `size()` devient coûteux.

2.8 Les tests `DynTabTest` (JUnit 4)

- `testAdd1` : suite d'`add(e)` avec contrôle de `size()` **et** de `capacity()` (1, 2, 4, 4, 8). C'est le test du doublement.
- `testAdd2` : `add(index, e)` à différentes positions, puis `indexOf` pour vérifier l'ordre final (C, D, E, A, B).
- `testAdd3`, `testAdd7` : indices invalides (-10 et 30). Voir 0.10 : assertion trop faible.
- `testAdd4/5/6` : insertion au début, au milieu, à la fin.
- `testRemoveInt3` : `remove(0)` sur vide doit lever `IndexOutOfBoundsException` (`assertTrue(false)` dans le `try` joue le rôle d'un `fail()`).
- `testGet` utilise `==` : correct ici car on compare **le même objet** `String`, mais `assertSame` serait plus clair.

3. TP3 – Tables associatives : `AssocTable` et table de hash fermée

Demandé (sujet TP3) : une table de hash **fermée** `ClosedHashDict<K,V>` qui implémente `java.util.Map`. Le fichier reçu pour ce chapitre est `AssocTable` : une table associative « naïve » (liste de couples, sans hash). C'est la base de comparaison idéale pour comprendre ce que le hash apporte. Les deux sont expliquées ci-dessous.

3.1 Le contrat `Map` en deux lignes

- Un `Map` associe à chaque **clé unique** une valeur. `put(k, v)` **renvoie l'ancienne valeur** associée à `k`, ou `null` s'il n'y en avait pas.
- `get(k)` renvoie la valeur ou `null` ; `remove(k)` renvoie la valeur retirée ou `null` ; `keySet()`, `values()`, `entrySet()` donnent trois vues de son contenu.

3.2 `AssocTable` : une `ArrayList` de couples

```
public class Assoc<K,V> implements Map.Entry<K, V>{
    K key;
    V value;
    public Assoc(K key, V value) {
        this.key = key;
        this.value = value;
    }
    @Override
    public K getKey() {
        return this.key;
    }
    @Override
    public V getValue() {
        return this.value;
    }
    @Override
    public V setValue(V value) {
        V prev = this.value;
        this.value = value;
        return prev;
    }

    public boolean equals(Object o) {
        if (o==null) return false;
        if (o==this) return true;
        if (! (o instanceof Assoc)) return false;
        @SuppressWarnings("unchecked")
        Assoc<K,V> a = (Assoc<K,V>) o;
        return a.getKey().equals(this.key);
    }
    public boolean equals(Assoc<K,V> o) {
        if (o==null) return false;
        if (o==this) return true;
        return o.getKey().equals(this.key);
    }

}
```

`Assoc` représente un couple et implémente `Map.Entry` (exigé par `entrySet`). Son `equals` ne compare **que la clé** : c'est le détail clé de la classe.

```
protected Assoc<K,V> getAssoc(K key) {
    Assoc<K,V> a = new Assoc<K,V>((K)key, null);
    int idx = assocs.indexOf(a);
    if (idx == -1) return null;
    return assocs.get(idx);
}
```

Pour chercher une clé, on fabrique un couple factice `(key, null)` et on demande à `ArrayList.indexOf` de le trouver : `indexOf` appelle `equals`, qui ne regarde que la clé. Astuce élégante, et `containsKey`, `get`, `remove` s'appuient dessus.

```
@Override
public V put(K key, V value) {
    Assoc<K,V> a = this.getAssoc((K) key);
    if (a != null) {
        V prev = a.getValue();
        a.setValue(value);
        return prev;
    }
    assocs.add(new Assoc<K,V>(key,value));
    return value;
}

@Override
public V remove(Object key) {
    Assoc<K,V> a = new Assoc<K,V>((K)key, null);
    int idx = assocs.indexOf(a);
    if (idx == -1) return null;
    V removed = assocs.get(idx).getValue();
    assocs.remove(idx);
    return removed;
}

@Override
public Set<K> keySet() {
    Set<K> s = new HashSet<K>();
    for (Assoc<K,V> a : assocs) {
        s.add(a.getKey());
    }
    return s;
}
```

Complexité

Tout est en **O(n)** : chaque recherche parcourt la liste. `size`, `isEmpty`, `clear` sont en O(1) (délégués à l'`ArrayList`). C'est ce que le hashage améliore.

Points à corriger

> **Bug vérifié.** `put` d'une **nouvelle** clé exécute `return value;` : il renvoie la valeur qu'on vient d'insérer. Le contrat `Map` demande `null`. Les tests ne le vérifient pas. Correction : `return null;`.

- `containsValue` plante (`NullPointerException`) si une valeur stockée est `null` : utiliser `Objects.equals(a.getValue(), value)`.
- `Assoc` redéfinit `equals` sans `hashCode` : règle violée (deux objets égaux doivent avoir le même hash). Sans conséquence ici car on ne s'en sert que via `indexOf`, mais `entrySet().contains(...)` ne marcherait pas.
- `keySet`, `values` et `entrySet` renvoient des **copies** : modifier le résultat ne modifie pas la table (une vraie implémentation renvoie des vues). Acceptable pour le TP, à connaître.
- `Assoc<K,V>` dans une classe déjà générique en `<K,V>` : les paramètres du `Assoc` **masquent** ceux de `AssocTable`. Plus simple : `private static class Assoc<K,V>`.

3.3 La table de hash fermée (principe du sujet)

Une table fermée est un **tableau de couples de taille fixe N**, sans listes. Pour une clé `k`, la position de départ est `p = Math.abs(k.hashCode()) % N`. On parcourt le tableau **de façon circulaire** : arrivé au bout, on repart de 0 jusqu'à revenir en `p`.

Opération	Parcours	Règle
`put(k, v)`	1er : chercher la clé. 2e (si absente) : chercher une case libre	Clé trouvée → remplacer la valeur. Sinon, 1re case `null` à partir de p. Retour à p sans case libre → **table pleine**
`get(k)`	1 seul	Retour à p sans trouver → clé absente (`null`)
`remove(k)`	1 seul	Case trouvée → remise à `null`

Exemple, N = 5, clés entières (`hashCode` d'un `Integer` = sa valeur). Les clés 3, 8 et 13 donnent toutes `p = 3` (collision) :

```
put(3,"a")  : p=3 libre                    [ _ ][ _ ][ _ ][3a][ _ ]
put(8,"b")  : p=3 occupée par la clé 3 ≠ 8, on passe à 4 :  [ _ ][ _ ][ _ ][3a][8b]
put(13,"c") : 3 puis 4 occupées, on repart à 0 :            [13c][ _ ][ _ ][3a][8b]
get(13)     : 3 → 4 → 0 trouvée
```

Piège. Après `remove(8)`, la case 4 redevient `null`. Si `get(13)` s'arrêtait à la première case vide rencontrée, il répondrait « absent » à tort (13 est en 0). C'est pourquoi le sujet demande de parcourir **jusqu'à revenir en** `p` et non de s'arrêter au premier `null`. Prix à payer : une recherche d'une clé absente coûte O(N).

Piège. `Math.abs(Integer.MIN_VALUE)` reste **négatif** (-2147483648) : `Math.abs(h) % N` donnerait alors un indice négatif et un `ArrayIndexOutOfBoundsException`. Plus sûr : `Math.floorMod(k.hashCode(), N)` (toujours dans [0, N-1]) ou `Math.abs(h % N)` comme dans `OpenedHashDict`.

À retenir. Contrairement à la table ouverte, la table fermée a une **capacité maximale** (N couples). Au-delà, `put` doit signaler que la table est pleine (choix de l'énoncé, par exemple une exception). Les tests `AssocTableTest` du chapitre suivant (même API `Map`) peuvent servir à tester `ClosedHashDict` : il suffit de changer la classe instanciée.

3.4 Les tests `AssocTableTest` (JUnit 5)

Classe et méthodes en visibilité package (pas de `public` obligatoire en JUnit 5). Chaque test construit son `AssocTable<Integer,Integer>` : `testClear` (taille, `isEmpty`, `containsKey` avant/après), `testContainsKey` / `testContainsValue` (présentes, absentes, puis après `remove`), `testGet` (valeurs puis `null` après `clear`), `testKeySet`, `testValues`, `testEntrySet` (parcours avec itérateur, recoupement avec des `HashSet`), `testPutAll`.

Piège. `testPutAll` appelle `containsKey` / `containsValue` **sans assertion** : il ne peut pas échouer sur ce point. Écrire `assertTrue(d2.containsKey(i))`. De plus `new Integer(i)` (dans `testGet`) est déprécié : `Integer.valueOf(i)` ou simplement `assertEquals(i, d.get(i))`.

4. TP4 – Table de hash ouverte : `OpenedHashDict`

Demandé : une table de hash **ouverte** : un tableau de taille fixe N dont chaque case contient une **liste** de couples. On peut donc stocker un nombre illimité de couples : les collisions s'empilent dans la liste. Clé `null` interdite.

```
table (N cases)
 0 → [(0,"a")] → [(100,"b")] → [(200,"c")]      ← trois clés avec p = 0 (N = 100)
 1 → null
 2 → [(2,"x")]
 ...
```

Sortie réelle de ton `toString()` avec `N = 100`, après `put(0,1)`, `put(100,2)`, `put(200,3)` :

```
(0): [([0][1]), ([100][2]), ([200][3])]
```

4.1 Structure et position

```
protected LinkedList<Couple<K,V>>[] table;        // tableau de listes (cases null = bucket vide)
protected Set<K> keysCache;  protected Collection<V> valuesCache;
protected Set<Map.Entry<K,V>> entriesCache;
protected boolean keyModified, valueModified;
protected final static int DEFAULT_CAPACITY = 100;

protected Couple<K, V> coupleFromKey(Object k) {
    int p = Math.abs(k.hashCode() % table.length);
    if (table[p] == null)
        return null;
    List<Couple<K, V>> v = table[p];
    for (Iterator<Couple<K, V>> i = v.iterator(); i.hasNext();) {
        Couple<K, V> c = i.next();
        if (c.getKey().equals(k))
            return c;
    }
    return null;
}
```

`p = Math.abs(k.hashCode() % table.length)` : le `%` est fait **avant** `Math.abs`, donc le résultat est dans [0, N-1] même pour `Integer.MIN_VALUE` (contrairement à `abs(h) % N`). Les listes sont créées **à la demande** (`table[p] == null` = aucun couple). Chercher une clé = aller à la case, puis parcourir la liste avec `equals`.

4.2 `put`, `get`, `remove`

```
@SuppressWarnings({ "unchecked", "rawtypes" })
public V put(K k, V v) {
    Couple<K, V> c = coupleFromKey(k);
    valueModified = true;
    if (c != null) {
        return (V) c.setValue(v);
    }
    int p = Math.abs(k.hashCode() % table.length);
    if (table[p] == null)
        table[p] = new LinkedList();
    table[p].add(new Couple(k, v));
    keyModified = true;
    return null;
}
```

- Clé déjà présente → `setValue` remplace la valeur et **renvoie l'ancienne** (contrat `Map` respecté). Sinon on crée la liste si besoin, on ajoute le couple, on renvoie `null`.
- Les booléens `keyModified` / `valueModified` servent à savoir si les caches doivent être reconstruits (voir 4.4).

```
public V remove(Object k) {
    Couple<K, V> c = coupleFromKey(k);
    if (c == null)
        return null;
    V oldValue = c.getValue();
    int p = Math.abs(k.hashCode() % table.length);
    table[p].remove(c);
    keyModified = true;
    valueModified = true;
    if (table[p].size() == 0)
        table[p] = null;
    return oldValue;
}
```

Après suppression, si la liste devient vide on remet la case à `null`. `table[p].remove(c)` retire l'objet `c` lui-même : `Couple` n'a pas de `equals`, c'est donc l'identité qui est utilisée, ce qui est exactement voulu ici.

Complexité

Avec N fixe et n couples : `put`/`get`/`remove` en O(1 + n/N) en moyenne (si le hash répartit bien), O(n) au pire (toutes les clés dans la même case). Choisir N proche du nombre de couples attendu garde les listes courtes. `size()` parcourt les N cases : O(N) ; un compteur tenu à jour la ferait passer à O(1).

4.3 Taille, parcours

```
public int size() {
    int s = 0;
    for (int n = 0; n < table.length; n++)
        if (table[n] != null)
            s += table[n].size();
    return s;
}
```

`isEmpty`, `containsValue`, `clear` suivent le même schéma : balayage des N cases en ignorant les `null`. `containsValue` doit tout parcourir (aucun hash sur les valeurs) : O(N + n).

4.4 Les vues `keySet`, `values`, `entrySet` et leurs caches

```
@SuppressWarnings({ "unchecked", "rawtypes" })
public Set<K> keySet() {
    if (keyModified) {
        keysCache = new HashSet();
        for (int n = 0; n < table.length; n++)
            if (table[n] != null)
                for (Iterator<Couple<K,V>> i = table[n].iterator(); i.hasNext();)
                    keysCache.add( (i.next()).getKey());
    }
    return keysCache;
}
```

Idée : ne reconstruire la vue que si la table a changé depuis le dernier appel. Les drapeaux sont mis à `true` par `put` (nouvelle clé), `remove`, `clear`, `putAll`.

Bug vérifié. Sur une table **neuve**, `keySet()`, `values()` et `entrySet()` renvoient `null` (exécuté) : les drapeaux valent `false` et le cache n'a jamais été construit. `new OpenedHashDict<Integer,Integer>().keySet().isEmpty()` plante avec une `NullPointerException`. Correction minimale : initialiser les drapeaux à `true` dans le constructeur.

À retenir. Les drapeaux ne sont **jamais remis à** `false` : après la première modification, les caches sont donc reconstruits à chaque appel (correct, mais le cache ne sert à rien). On ne peut pas simplement les remettre à `false` dans `keySet()` : `entrySet()` utilise les mêmes drapeaux et recevrait un cache périmé. Il faudrait **un drapeau par cache**. Plus simple et sûr : supprimer les caches et construire la vue à chaque appel :

```
public Set<K> keySet() {
    Set<K> keys = new HashSet<K>();
    for (LinkedList<Couple<K,V>> bucket : table)
        if (bucket != null)
            for (Couple<K,V> c : bucket) keys.add(c.getKey());
    return keys;
}
```

4.5 Autres remarques

- `Couple` redéfinit `setValue(Object)` (type brut) au lieu de `V setValue(V)` : ça compile, mais `V setValue(V value)` est la signature correcte de `Map.Entry`.
- `containsValue` : `c.getValue().equals(v)` plante si une valeur vaut `null` → `Objects.equals`.
- `putAll(Map t)` utilise le type brut `Map` (d'où les `@SuppressWarnings`) ; la signature générique est `putAll(Map<? extends K, ? extends V> m)`.
- La clé `null` n'est pas filtrée : `k.hashCode()` lève une `NullPointerException`. Cohérent avec l'énoncé, mais un message explicite serait mieux.

4.6 Les tests `OpenedHashDictTest` (JUnit 5)

Ce sont **exactement les mêmes sept tests** qu'`AssocTableTest` : les deux classes respectent la même interface `Map`, donc le même jeu de tests les valide. C'est le principe du test d'une interface. En revanche, rien ne teste ce qui fait la spécificité d'une table de hash. Trois tests à ajouter (le troisième échoue aujourd'hui, il met en évidence le bug 4.4) :

```
@Test
void collisions() {
    OpenedHashDict<Integer,String> d = new OpenedHashDict<Integer,String>(100);
    d.put(0, "a"); d.put(100, "b"); d.put(200, "c");     // même case : p = 0
    assertEquals(3, d.size());
    assertEquals("b", d.get(100));
    assertEquals("b", d.remove(100));
    assertNull(d.get(100));
    assertEquals("c", d.get(200));                       // le reste de la liste est intact
}

@Test
void putRetourneAncienneValeur() {
    OpenedHashDict<Integer,String> d = new OpenedHashDict<Integer,String>();
    assertNull(d.put(1, "a"));
    assertEquals("a", d.put(1, "b"));
    assertEquals(1, d.size());
}

@Test
void keySetSurTableVide() {
    assertTrue(new OpenedHashDict<Integer,Integer>().keySet().isEmpty());   // rouge tant que non
        corrigé
}
```

5. TP5 – Ensemble : `MonEnsemble` et `MonIterateur`

Demandé : un ensemble (collection sans doublon, non ordonnée) stocké dans un tableau d'objets où des cases peuvent être libres (`null`) ; compléter `MonEnsemble` et `MonIterateur`, puis (exercice 2) faire lever une `ConcurrentModificationException` quand on modifie l'ensemble pendant un parcours.

5.1 Représentation

```
protected Object[] tab;          // les éléments ; null = case libre
protected int nbPlacesLibres;    // nombre de cases null
protected int version;           // compteur de modifications (pour l'exercice 2)

tab : [null][1][8][null][null][-3][9]    nbPlacesLibres = 3   →   {1, 8, -3, 9}
```

Invariant : `size() = capacity() - nbPlacesLibres`, où `capacity() = tab.length` ; la taille n'est donc jamais stockée, elle est déduite. Le constructeur crée un tableau de 10 cases toutes libres.

5.2 `contains`, `add`, `grow`

```
@Override
public boolean contains(Object o) {
    // TODO à compléter...
    for (Object e : tab) {
        if (e != null && e.equals(o)) {
            return true;
        }
    }
    return false; // <- TODO résultat à mettre à jour
}

protected void grow(int gap) {
    Object [] prevTab = tab;
    tab = new Object[this.capacity() + gap];
    System.arraycopy(prevTab, 0, tab, 0, prevTab.length);
    this.nbPlacesLibres += gap;
}

@Override
public boolean add(E e) {
    // TODO à compléter...
    if (this.contains(e)) return false;
    this.version++;
    if (this.nbPlacesLibres == 0) grow(this.capacity());
    for (int i = 0; i < this.capacity(); i++) {
        Object curr = this.tab[i];
        if (curr == null) {
            this.tab[i] = e;
            this.nbPlacesLibres--;
            return true;
        }
    }
    throw new Error("This should not happen");
    // return false; // <- TODO résultat à mettre à jour
}
```

`add` : (1) si déjà présent → `false`, rien ne change ; (2) `version++` ; (3) plus de place → `grow(capacity())` double la capacité (copie par `System.arraycopy`, `nbPlacesLibres += gap`) ; (4) la **première case** `null` reçoit l'élément. Le `throw new Error` ne doit jamais être atteint : il documente l'invariant.

Conséquence à connaître : l'ordre des éléments est l'**ordre des cases**, pas celui d'insertion. Après `{1,2,3,4}`, `remove(2)`, `add(9)`, on obtient `(1, 9, 3, 4)` : 9 prend la case libérée.

Coût : `add` = O(capacité) (le `contains` puis la recherche de case libre), amorti sur les agrandissements.

5.3 `remove`, `toArray`, `compact`, `toString`

```
@Override
public boolean remove(Object o) {
    // TODO à compléter...
    this.version++;
    for (int i = 0; i < this.capacity(); i++) {
        Object curr = this.tab[i];
        if (curr != null && curr.equals(o)) {
            this.tab[i] = null;
            this.nbPlacesLibres++;
            return true;
        }
    }
    return false; // <- TODO résultat à mettre à jour
}
```

On cherche l'élément, on **remet sa case à** `null` et `nbPlacesLibres++` ; le tableau n'est pas recréé (comme demandé).

```
@Override
public Object[] toArray() {
    // TODO à compléter...
    Object [] result = new Object[this.size()];
    int idx = 0;
    for (int i = 0; i < this.capacity(); i++) {
        Object curr = this.tab[i];
        if (curr != null) {
            result[idx++] = curr;
        }
    }
    return result;
    //return null; // <- TODO résultat à mettre à jour
}

public void compact() {
    // Recrée le tableau d'objets sans place disponible
    this.tab = this.toArray();
    this.nbPlacesLibres = 0;
    this.version++;
    // TODO à compléter...
}
```

`toArray` produit un tableau de la taille exacte, sans les `null`. `compact` s'en sert pour recréer un tableau **sans case libre** (`nbPlacesLibres = 0`) et incrémente `version` : les indices ont changé, un itérateur en cours devient invalide, c'est voulu.

```
@Override
public String toString() {
    // TODO à compléter...
    if (this.isEmpty()) return "()";
    StringBuffer sb = new StringBuffer();
    sb.append('(');
    for (int i = 0; i < this.capacity(); i++) {
        Object curr = this.tab[i];
        if (curr != null) {
            sb.append(curr.toString());
            sb.append(", ");
        }
    }
    sb.delete(sb.length()-2, sb.length());
    sb.append(')');
    return sb.toString();

    // return null; // <- TODO résultat à mettre à jour
}
```

Format attendu par `testToString` : `(1, 2, 3, 4)` et `()` pour un ensemble vide. On ajoute « , » après chaque élément puis on supprime les deux derniers caractères avant de fermer la parenthèse (d'où le cas vide traité à part). `StringBuilder` suffit (`StringBuffer` est synchronisé, donc plus lent, inutile ici).

5.4 Trois bugs vérifiés dans `MonEnsemble`

Bug vérifié. `add` **après** `compact()` **sur un ensemble vide** : `compact()` laisse `tab.length == 0` et `nbPlacesLibres == 0` ; `grow(0)` n'ajoute rien, aucune case n'est libre et `add` lève `java.lang.Error: This should not happen` (exécuté). Correction : `grow(Math.max(1, this.capacity()))`.

Bug vérifié. `add(null)` : `contains(null)` renvoie `false` (le test ignore les `null`), donc `add(null)` « réussit » : il écrit `null` dans une case qui reste libre, mais décrémente `nbPlacesLibres`. Résultat exécuté : `size() == 1` avec aucun élément. Correction : refuser `null` (`throw new NullPointerException()`) ou renvoyer `false`.

Bug vérifié. `remove` **incrémente** `version` **même si l'élément est absent** : un `remove(99)` inutile fait lever `ConcurrentModificationException` à un itérateur en cours (exécuté). Correction : déplacer `version++` juste avant `return true`.

5.5 `MonIterateur` et l'exercice 2

```
@Override
public boolean hasNext() {
    for (int i = currentIdx + 1; i < src.capacity(); i++) {
        if (src.tab[i] != null) return true;
    }
    return false;
}

@SuppressWarnings("unchecked")
@Override
public E next() {
    if (this.localVersion == -1) {
        this.localVersion = src.version();
    } else if (this.localVersion != src.version()) {
        throw new ConcurrentModificationException();
    }
    do {
        currentIdx++;
    } while (src.tab[currentIdx] == null);
    return (E) this.src.tab[currentIdx];
}
```

- L'itérateur garde `currentIdx` (dernière case renvoyée, -1 au départ) et `localVersion`. `hasNext` cherche une case non `null` après `currentIdx` ; `next` avance jusqu'à la prochaine.
- **Principe fail-fast** (exercice 2) : `MonEnsemble` incrémente `version` à chaque modification structurelle (`add` réussi, `remove`, `compact`). L'itérateur mémorise la version vue ; si elle a changé au prochain `next()`, il lève `ConcurrentModificationException`. C'est le même mécanisme (`modCount`) que `HashSet` et `ArrayList`.
- **Pourquoi la version est lue au premier** `next()` **et pas à la création ?** Parce que `MonIterateurTest.test` crée l'itérateur sur un ensemble **vide**, y ajoute ensuite 4 éléments, puis le parcourt : capturer la version dans le constructeur ferait lever l'exception à tort. Compromis : une modification entre la création et le premier `next()` n'est pas détectée (le standard Java, lui, capture à la création).

Bug vérifié. `next()` quand il n'y a plus d'élément lève `ArrayIndexOutOfBoundsException` (exécuté) ; le contrat `Iterator` demande `NoSuchElementException`. Correction :

```
public E next() {
    if (!hasNext()) throw new java.util.NoSuchElementException();
    if (this.localVersion == -1) this.localVersion = src.version();
    else if (this.localVersion != src.version()) throw new ConcurrentModificationException();
    do { currentIdx++; } while (src.tab[currentIdx] == null);
    return (E) this.src.tab[currentIdx];
}
```

5.6 Les tests (JUnit 4)

`MonEnsembleTest.test1` : cycle complet add / contains / remove avec `size` et `isEmpty` à chaque étape, y compris le doublon (`add(2)` deux fois → deuxième `false`). `testToString` : `(1, 2, 3, 4)` puis `()` après suppression de tout. `MonIterateurTest.test` : itérateur créé avant les ajouts, parcours complet, comparaison des tailles.

> **Bug vérifié.** Le test `concurrentModificationExceptionTest` du fichier reçu ne **termine pas** : le `try/catch` est à l'intérieur de la boucle. Après la première exception, l'itérateur n'a pas avancé, `hasNext()` reste vrai, `next()` relance la même exception, et ainsi de suite. Exécuté avec un plafond de 10 tours : 9 exceptions attrapées, la boucle n'aurait jamais fini. Version correcte (JUnit 4.13+ ; sinon `@Test(expected = ConcurrentModificationException.class)` sur la méthode, sans `try/catch`) :

```
@Test
public void concurrentModificationExceptionTest() {
    MonEnsemble<Integer> ens = new MonEnsemble<Integer>();
    for (int i = 1; i <= 4; i++) ens.add(i);
    Iterator<Integer> itor = ens.iterator();
    assertThrows(ConcurrentModificationException.class, () -> {
        while (itor.hasNext())
            ens.add(itor.next() + 10);      // 1er next OK ; add change la version ; 2e next lève
                l'exception
    });
}
```

Test à ajouter pour `compact` (le test actuel ne l'appelle pas) :

```
@Test
public void testCompactVraiment() {
    MonEnsemble<Integer> ens = new MonEnsemble<Integer>();
    for (int i = 1; i <= 4; i++) ens.add(i);
    ens.remove(2);
    ens.compact();
    assertEquals(3, ens.capacity());              // plus aucune case libre
    assertEquals(0, ens.nbPlacesLibres);
    assertEquals("(1, 3, 4)", ens.toString());
}
```

6. TP6 – File circulaire : `ArrayQueue`

> **Non fourni.** Le fichier `ArrayQueue.java` n'est pas dans ce que tu as envoyé : seuls les interfaces, `ArrayQueueTest` et `ArrayQueueMain` y sont. Ce chapitre explique donc le sujet, les interfaces et **ce que les tests imposent**, avec un squelette pour t'orienter (à comparer avec ton code).

Demandé : une file FIFO (premier entré, premier sorti) à **capacité fixe**, stockée dans un tableau géré de façon **circulaire**. Ajout en queue, retrait en tête. Ajout dans une file pleine → `IllegalStateException`.

6.1 Les deux interfaces

Rôle	Lève une exception	Renvoie une valeur spéciale
Ajouter	`add(e)` : `IllegalStateException` si pleine	`offer(e)` : `false` si pleine
Retirer la tête	`remove()` : `NoSuchElementException` si vide	`poll()` : `null` si vide
Lire la tête	`element()` : `NoSuchElementException` si vide	`peek()` : `null` si vide

`SimpleQueue` ajoute `size`, `isEmpty`, `clear`, `iterator` et `last()` (dernier élément, exception si vide). `RestrictedSimpleQueue` couvre la capacité : `capacity()`, `isFull()`, `enlargedTo(supplement)`. On retrouve la convention de `java.util.Queue`.

6.2 Le tableau circulaire (table 1 du sujet)

```
capacité 4 :  q.add(1); q.add(2); q.add(3);      indices :  0  1  2  3
                                                  contenu : [1][2][3][ ]     tête = 0, queue = 2
              q.add(4);                                     [1][2][3][4]     tête = 0, queue = 3
                  (pleine)
              q.remove();                                   [ ][2][3][4]     tête = 1, queue = 3
              q.add(5);   ← on « revient » au début          [5][2][3][4]     tête = 1, queue = 0
                  (pleine)
```

`tete` = indice du **plus ancien** élément (prochain à sortir) ; `queue` = indice du **dernier ajouté**. On avance avec `(indice + 1) % capacité` : c'est ce `%` qui fait « revenir » au début du tableau. Ce sont ces valeurs que `addRemoveIsFull` contrôle (`tete() == 0`, `queue() == 2` après trois ajouts).

6.3 Ce que les tests imposent

- `null` **est un élément valide** (`destrucs` contient `null`, `add1` empile six `null`). On ne peut donc pas utiliser `null` pour reconnaître une case libre : il faut un **compteur** (ou comparer `tete` et `queue` avec une case de plus). C'est la contrainte la plus importante du TP.
- `peek()` renvoie `null` sur une file vide, mais `peek2` vérifie aussi `peek() == remove()` pour un élément `null` : les deux cas se confondent pour `peek`, d'où l'importance de `element()` qui lève une exception.
- `last()` sur une file vide → `NoSuchElementException` ; sur une file dont le dernier élément est `null` → renvoie `null`.
- Après `clear()` : l'état est celui d'une file neuve (`last()` et `element()` relèvent `NoSuchElementException`).
- `enlargedTo(n)` : la capacité devient `capacity + n`, **le contenu et l'ordre sont conservés** (`enlargedTo2` compare, élément par élément, l'itérateur de la file agrandie avec une copie non agrandie).
- `iterator()` est **« vivant »** : créé sur une file vide, il voit les éléments ajoutés ensuite (`iterator1`, `iterator2`) ; comme `MonIterateur`, il ne fige pas son état à la création.

6.4 Squelette possible (mon illustration, pas ton code)

```
// champs : tableau, tete, queue (protected, lus par ArrayQueueTest) + un compteur taille
tete = 0;  queue = capacity - 1;  taille = 0;          // état initial : le 1er ajout ira en case 0

add(e):    if (taille == capacity) throw new IllegalStateException();
           queue = (queue + 1) % capacity;  tableau[queue] = e;  taille++;  return true;
remove():  if (taille == 0) throw new NoSuchElementException();
           E e = tableau[tete];  tableau[tete] = null;  tete = (tete + 1) % capacity;  taille--;
               return e;
element(): if (taille == 0) throw new NoSuchElementException();  return tableau[tete];
last():    if (taille == 0) throw new NoSuchElementException();  return tableau[queue];
isFull():  return taille == capacity;
clear():   remettre l'état initial et vider le tableau
enlargedTo(sup): nouveau tableau de taille capacity + sup ; copier les `taille` éléments
           dans l'ordre depuis `tete` (avec le %) vers les cases 0..taille-1 ; tete = 0 ; queue =
               taille - 1
```

Vérification avec la table 1 : état initial `queue = 3`. Premier `add` → `queue = 0`… trois ajouts → `queue = 2` ✓ ; quatrième → 3 ✓ ; `remove` → `tete = 1` ✓ ; `add(5)` → `queue = (3+1) % 4 = 0` ✓.

6.5 `ArrayQueueTest` (JUnit 5) : lecture rapide

- **Mise en place** : la sous-classe `ArrayQueue` dans le package `tests` expose `tete()`, `queue()`, `tableau()` (voir 0.7) ; `myq` est recréé pour chaque test (0.5).
- `size`, `isEmptyIsFull`, `isFull` : cohérence du compteur lors d'ajouts / retraits, y compris le passage plein → non plein.
- `add` : après chaque ajout, `element()` est le premier ajouté et `last()` le dernier. `add1` : six `null` puis un entier ressortent dans l'ordre. `add2` : `assertThrows(IllegalStateException.class, …)` sur file pleine. `offer` : `false` quand pleine.
- `remove`, `peek`, `last` : exceptions sur file vide, valeurs sinon. `clear`, `iterator1/2`, `enlargedTo`, `enlargedTo2` : voir 6.3.
- `enlargedTo2` compare avec `==` des `Integer` : valable car ce sont les **mêmes objets** (tableau `destrucs`), pas par hasard de cache.
- `ArrayQueueMain` (hors JUnit) : petit programme de démonstration avec `main` ; ne remplace pas les tests.

Rappel du sujet : « tous les tests doivent passer au vert ». Attention : `size` contient la boucle morte signalée en 0.10, qui ne vérifie rien.

7. Récapitulatif des points à corriger

TP	Fichier	Problème (vérifié)	Correction
1	`CharArray.containsSub`	`"ABAC"` / `"AC"` → `false` : seule la 1re occurrence du 1er caractère est testée	Boucler sur toutes les positions de départ
1	`CharArray.toString`	`s +=` en $O(n^2)$	`StringBuilder`
2	`DynTab.add`	Recopie du début à chaque appel, même sans réallocation : O(n)	`System.arraycopy`, recopie du début seulement si le tableau a changé
2	`DynTab.equals`	Non redéfini → égalité d'identité	`equals` + `hashCode` élément par élément
2	`DynTab.remove(int)`	Référence conservée en `tab[s]` (fuite mémoire)	`tab[s] = null` après `s--`
3	`AssocTable.put`	Renvoie la valeur insérée au lieu de `null`	`return null;`
3	`AssocTableTest.testPutAll`	Aucune assertion sur les résultats	`assertTrue(d2.containsKey(i))`
4	`OpenedHashDict.keySet/values/entrySet`	`null` sur une table neuve	Drapeaux à `true` au départ, ou pas de cache
5	`MonEnsemble.add`	`add` après `compact()` sur ensemble vide → `Error`	`grow(Math.max(1, capacity()))`
5	`MonEnsemble.add`	`add(null)` corrompt la taille	Refuser `null`
5	`MonEnsemble.remove`	`version++` même si absent	`version++` seulement en cas de succès
5	`MonIterateur.next`	`ArrayIndexOutOfBoundsException` en fin de parcours	`NoSuchElementException`
5	`MonIterateurTest`	Test CME : boucle infinie	`assertThrows` autour de la boucle
5	`MonEnsembleTest.testCompact`	N'appelle pas `compact()`	Test proposé en 5.6
6	`ArrayQueueTest.size`	Boucle jamais exécutée	Réécrire la boucle de retrait

Ces points ne remettent pas en cause tes corrigés : ils passent les tests fournis. Mais ils montrent où les tests sont trop gentils, c'est utile à savoir avant un contrôle.